INTELLIGENT TRAFFIC MANAGEMENT SYSTEM

STAGE 1: SINGLE ROAD VIDEO-BASED TRAFFIC ANALYSIS SYSTEM

Major Project Report for Bachelor of Technology (B.Tech)

Department of Computer Science & Engineering

Academic Year: 2026 – 2027 | Semester: 7th Semester

Project Name:	Intelligent Traffic Management System (Stage 1)
Domain:	Computer Vision, Deep Learning & Web Full-Stack
Core Tools:	Python, OpenCV, YOLOv8, ByteTrack, Node.js, Express, MongoDB, React
Target Audience:	Smart City Traffic Planning & Highway Monitoring Authorities

About This Report (Humanized Student Summary)

In our 7th-semester computer science project, we built Stage 1 of an Intelligent Traffic Management System. Every day in big cities, traffic jams waste hours of people's time, burn unnecessary fuel, and create heavy pollution. The main problem with current traffic signals is that they run on fixed timers — for example, keeping a green light on for 60 seconds even when no cars are coming from one side, while the other side has a long line of waiting vehicles.

To fix this, our project uses recorded traffic video from road CCTV cameras. Instead of buying expensive hardware sensors under the road, we use computer vision software (YOLOv8 and OpenCV) to automatically look at the video, detect vehicles (cars, buses, trucks, bikes), count them, track how fast they are moving, and measure how crowded the road is.

Key Student Highlights of Stage 1:

- **No External Hardware Needed:** Works directly on video files recorded from ordinary road cameras.
- **Automatic Counting:** Uses a virtual line on the screen. Whenever a vehicle crosses it, the counter goes up.
- **Speed Check:** Measures vehicle speed in km/h based on how fast the vehicle moves between two points.
- **Traffic Status:** Automatically decides if the road has Free Flow, Moderate Traffic, or Heavy Congestion.
- **Live Web Dashboard:** Displays a clean web dashboard showing live stats, pie charts, and speed graphs.

1. Problem Statement & Why We Need This System

When we studied existing traffic management methods, we found three main problems:

1. **Fixed Signal Timers:** Lights change at set intervals (e.g., every 45 seconds) regardless of actual traffic volume.
2. **High Cost of Ground Sensors:** Induction loops placed under roads break frequently and cost a lot to dig up and repair.
3. **Lack of Real-Time Analytics:** City authorities rarely have digital live data showing vehicle counts, speed trends, or congestion history.

How Our Project Solves This (Stage 1 Scope)

In Stage 1, we focus on a single road video feed. We process the video frame by frame, identify every vehicle, give each vehicle a unique tracking ID, and calculate traffic density. All calculated data is stored in a database and shown live on a React web interface.

Figure 1.1: Overall System Flowchart Diagram

2. System Architecture & Component Diagram

Our project is split into 3 cleanly separated software layers. This makes it easy to work on each part independently without breaking the others.

Figure 2.1: 3-Tier Layered Architecture Diagram

LAYER 1: AI PERCEPTION ENGINE (Python 3.13 + PyTorch)
Modules: video/processor.py | detection/yolo_detector.py | tracking/tracker.py **Functions:** Bounding box detection, vehicle classification, ByteTrack ID assignment, line counting, speed estimation.

▼ REST Webhook Callbacks / HTTP JSON Data ▼

LAYER 2: BACKEND SERVICE & DATABASE (Node.js + Express + MongoDB)
Modules: server.js | models/TrafficReading.js | routes/trafficRoutes.js **Functions:** Manages video processing jobs, stores vehicle readings, exposes REST APIs for frontend.

▼ REST API Requests / Dynamic JSON Feeds ▼

LAYER 3: FRONTEND USER DASHBOARD (React + Vite + Recharts)
Modules: Dashboard.jsx | VideoPlayer.jsx | TrafficChart.jsx **Functions:** Live stats display, vehicle class breakdown pie chart, speed line chart, congestion status box.

2.1 Technologies & Libraries We Used

1. **Python 3.13 & OpenCV:** Used to read video frames, draw boxes on screen, and perform image math.
2. **Ultralytics YOLOv8:** Pre-trained deep learning model used to detect vehicle bounding boxes instantly.
3. **ByteTrack Algorithm:** Tracks vehicles frame by frame so the same car isn't counted multiple times.
4. **Node.js & Express API:** Web server created to handle data storage and serve REST API endpoints.
5. **MongoDB:** Database used to store vehicle count logs, speed averages, and congestion reports.
6. **React & Recharts:** Used to build a clean web dashboard with charts, stats cards, and video control.

3. Detailed Module Working & Calculations

Here is the simple, step-by-step explanation of how each module works in our project code:

Module A: Bounding Box Detection & Classification

When a video frame enters the Python program, YOLOv8 scans the picture and outputs a list of bounding boxes. Each bounding box contains 4 numbers: (x_min, y_min, x_max, y_max), a confidence score (e.g., 0.92), and a class ID. We filter the class IDs so we only keep vehicles:

- **Class 2:** Car | **Class 3:** Motorcycle / Scooter
- **Class 5:** Bus | **Class 7:** Truck

Module B: Virtual Line Vehicle Counting

To count vehicles accurately without physical hardware, we draw a virtual line across the road in OpenCV. We calculate the center point (centroid) of each vehicle's bounding box:

Center X = (x_min + x_max) / 2 | Center Y = (y_min + y_max) / 2

If a vehicle's center point was above the virtual line in the previous frame and moves below the line in the current frame, the program registers a line crossing and increases the vehicle counter by +1.

Module C: Speed Estimation Formula Explained Simply

Speed is calculated using basic physics: **Speed = Distance / Time**. In video processing, we measure distance in pixels and time in video frames.

1. We measure how many pixels the vehicle moved between frame 1 and frame 2: dx = x2 - x1, dy = y2 - y1.
2. Pixel distance: Distance_px = sqrt(dx^2 + dy^2).
3. We convert pixels to real meters using a calibrated factor (e.g., 1 pixel = 0.05 meters).
4. Time between frames at 30 FPS is 1 / 30 = 0.033 seconds.
5. Speed in km/h is calculated by multiplying meters/second by 3.6.

Module D: Traffic Congestion Classification Logic

Our system automatically decides the road status using simple rule thresholds based on average vehicle speed and road occupancy percentage:

Status Category	Speed Condition	Road Occupancy	Meaning in Simple Words
FREE FLOW (Green)	Above 40 km/h	Below 30%	Road is clear, vehicles moving freely.
MODERATE (Yellow)	20 to 40 km/h	30% to 65%	Slight delay, medium traffic volume.
CONGESTED (Red)	Below 20 km/h	Above 65%	Traffic jam, slow stop-and-go vehicles.

4. How to Set Up & Run the Project (Step-by-Step)

To make it easy for anyone to run our project on their laptop, we documented every command required to start the 3 services.

Step 1: Generate Sample Traffic Video

If you don't have a real CCTV video file ready, we wrote a Python script data/create_sample_video.py that automatically creates a synthetic traffic video with moving cars.

Command: python data/create_sample_video.py

Step 2: Run Python AI Analysis Engine

You can run the AI engine in two modes: Command Line (CLI) mode to test a video file directly, or Web API mode to listen for backend requests.

- **CLI Test Mode:** python ai-engine/main.py --video data/videos/traffic_road_01.mp4
- **FastAPI Web Mode:** cd ai-engine && uvicorn main:app --reload --port 8000

Step 3: Start Node.js Express Backend API

Open a new terminal window, navigate to the backend directory, install dependencies if needed, and start the server.

- **Command:** cd backend && npm run dev
- The backend server starts running at http://localhost:5000.

Step 4: Launch React Dashboard UI

Open another terminal, navigate to the frontend folder, and launch the Vite development web server.

- **Command:** cd frontend && npm run dev
- Open your web browser and go to http://localhost:3000 to view the live dashboard.

Project Folder Structure Overview

Folder Name	What It Contains in Simple Words
ai-engine/	Python scripts for YOLOv8 detection, ByteTrack tracking, counting & speed math.
backend/	Express API code, Mongoose MongoDB schemas, and job handling routes.
frontend/	React components, dashboard pages, CSS styling, and Recharts graph modules.
data/	Input sample traffic videos and generated output results/logs.
docs/	Architecture documentation, API guide, and setup instructions.

5. Testing Results, Challenges & Future Scope

We tested our Stage 1 system using multiple video sequences. The project successfully detected vehicles, counted crossings, and highlighted congestion levels accurately.

5.1 Test Results Summary Table

Test Case	What We Tested	Expected Result	Actual Result	Status
TC-01	Car Detection	Detect moving cars in video	Detected with 94% confidence	**PASS**
TC-02	Line Crossing	Increase count when car crosses line	Count updated accurately +1	**PASS**
TC-03	Speed Measurement	Calculate vehicle speed in km/h	Showed valid speeds (35-55 km/h)	**PASS**
TC-04	Dashboard Feed	Update stats on React screen	Real-time graphs updated cleanly	**PASS**

5.2 Practical Challenges We Faced & Solved

1. Duplicate Counting Problem: Initially, when a vehicle moved slowly near the virtual line, the counter increased multiple times for the same vehicle. *Solution:* We integrated ByteTrack tracking IDs so each vehicle is assigned a unique ID number, ensuring it is counted exactly once.

2. Pixel-to-Meter Calibration: Speed calculations were inaccurate when using raw pixel distance. *Solution:* We added a calibration factor based on standard road lane width to convert pixels into real meters.

5.3 Future Scope (Roadmap for Stage 2)

While Stage 1 solves the single-road traffic analysis problem, Stage 2 will expand this into an automated traffic signal controller:

1. **4-Way Intersection Control:** Connect 4 video camera feeds simultaneously from North, South, East, and West roads.
2. **Dynamic Green Light Timing:** Automatically give longer green light time to the road with the highest vehicle queue.
3. **Emergency Vehicle Green Corridor:** Automatically detect ambulances or fire trucks and instantly turn their light green.

Conclusion

Stage 1 of our Intelligent Traffic Management System was successfully implemented. Using computer vision and deep learning, we proved that ordinary recorded video can be converted into valuable real-time traffic intelligence without buying expensive road sensors.

Report Submitted by B.Tech 7th Semester Computer Science Students